

## 第5章 设备管理

- 5.1 设备管理概述
- 5.2 设备控制方式
- 5.3 I/O设备管理系统的组成
- 5.4 I/O设备控制方式
- 5.5 生磁盘的使用
- 5.6 从生磁盘到文件
- 5.7 设备的分配与回收
- 5.8 磁盘存储器管理

## 5.1 设备管理概述

设备又称I/O（输入/输出）设备，是指计算机系统中除CPU和内存之外的所有外部设备。

- 设备的分类

- 1. 按使用特性分类
  - (1) 存储设备:外存或辅助存储器，用于永久保存用户使用计算机处理的信息。磁盘、磁带、光盘和优盘都属于存储设备。
  - (2) I/O设备:键盘、扫描仪、显示器、打印机等。
- 2. 按信息传输速率分类
  - (1) 低速设备。指传输速率为每秒几字节至几百字节的设备，如键盘、鼠标等。
  - (2) 中速设备。指传输速率为每秒几千字节至几万字节的设备，如各种打印机。
  - (3) 高速设备。指传输速率为每秒几十万字节甚至几兆字节的设备，如闪存（优盘）、光盘机、磁盘机及磁带机等。

## 设备的分类

### 3. 按信息传输单位分类

- (1) 字符设备：打印机、键盘、显示终端；
- (2) 块设备：磁盘。

### 4. 按资源分配方式分类

- (1) 独占设备
- (2) 共享设备
- (3) 虚拟设备

### 5. 按设备的从属关系分类

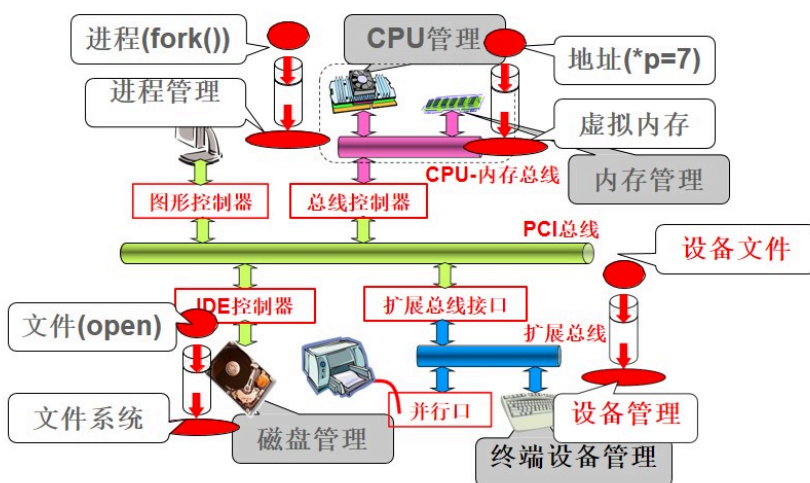
- (1) 系统设备
- (2) 用户设备

## 5.1 设备管理概述

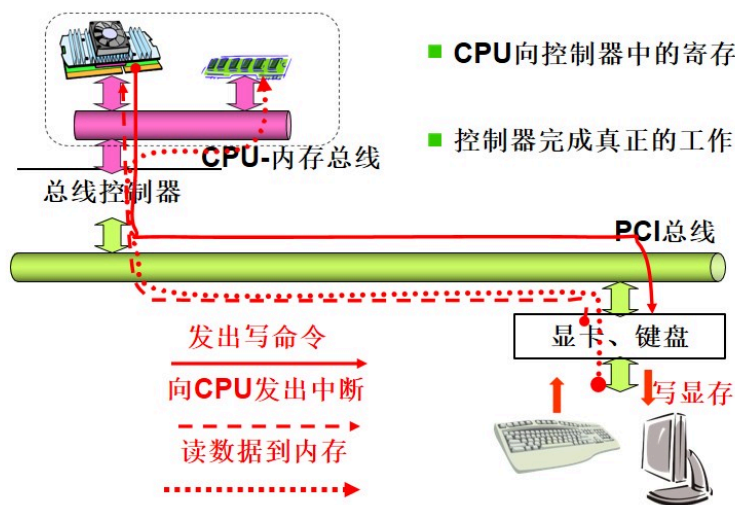
- 设备管理的目标和功能
  - 1. 设备管理的目标
  - 计算机配置操作系统的主要目的，一是为了提高系统资源的利用率，二是方便用户使用计算机。
  - 设备管理的目标有：
    - (1) 方便性
    - (2) 并行性
    - (3) 均衡性
    - (4) 独立性
  - 2. 设备管理的功能
    - (1) 提供设备使用的接口
    - (2) 进行设备的分配与回收
    - (3) 缓冲区管理
    - (4) 实现物理I/O设备的操作（真正的I/O操作）
    - (5) 设备的访问和控制

## 5.2 设备控制方式

继续那台“计算机”



## 让外设工作起来



■ CPU向控制器中的寄存器 读写数据

■ 控制器完成真正的工作，并向CPU发中断信号

## 向设备控制器的寄存器写不就可以了吗？

需要查寄存器地址、内容的格式和语义...操作系统要给用户提供一个简单视图—**文件视图**，这样方便

## 一段操纵外设的程序

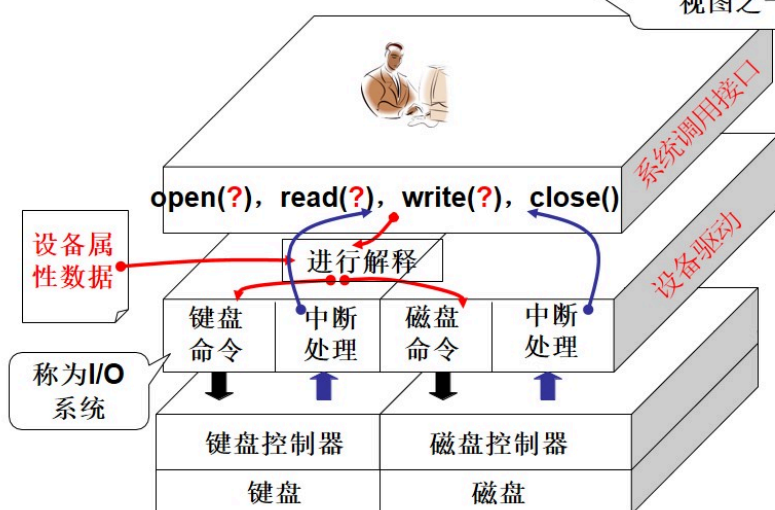


```
int fd = open("/dev/xxx");  
for (int i = 0; i < 10; i++) {  
    write(fd, i, sizeof(int));  
}  
close(fd);
```

- (1) 不论什么设备都是 **open, read, write, close**  
操作系统为用户提供统一的接口!
- (2) 不同的设备对应不同的设备文件(/dev/xxx)  
根据设备文件找到控制器的地址、内容格式等等!

## 一个统一的视图-文件视图

操作系统两大视图之一



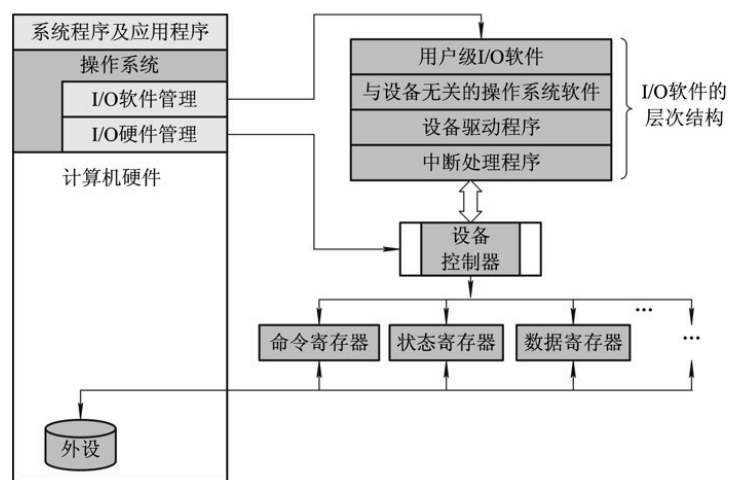
**单选题** 1分

用户程序发出磁盘I/O请求后，系统的正确处理流程是\_\_\_\_\_。

- ☐ A 用户程序→系统调用处理程序→中断处理程序→设备驱动程序
- ☐ B 用户程序→系统调用处理程序→设备驱动程序→中断处理程序
- ☐ C 用户程序→设备驱动程序→系统调用处理程序→中断处理程序
- ☐ D 用户程序→设备驱动程序→中断处理程序→系统调用处理程序

## 5.3 I/O设备管理系统的组成

- I/O设备管理系统的结构图

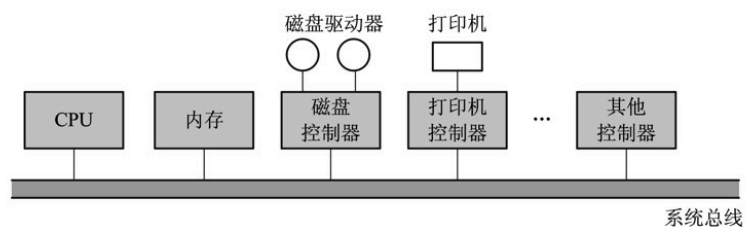


- I/O系统的硬件组织

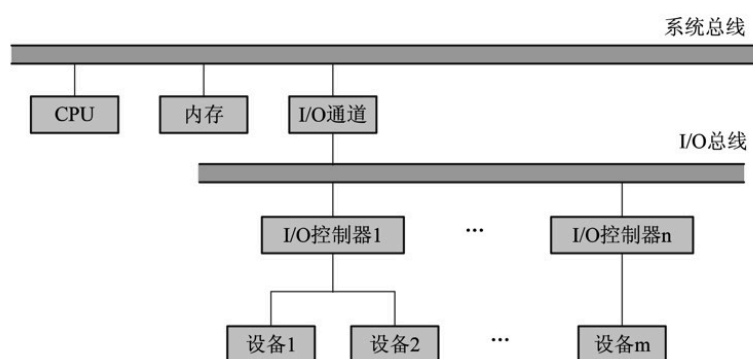
- 1. I/O系统的硬件结构

- (1) 单总线型I/O系统

- CPU和内存直接连到系统总线，设备通过控制器连到系统总线。微型计算机等简单系统，CPU和设备控制器进行通信。设备控制器是控制一个或多个IO设备的硬件，例如磁盘控制器，打印机控制器，显示器控制器（显卡，适配器）。



- (2) 大型机I/O系统
- IO设备较多时，增加一级IO通道代替CPU与设备控制器进行通信。



I/O系统共分四级：最低级为I/O设备，次低级为设备控制器，次高级为I/O通道，最高级是CPU。

## • 2. 设备控制器

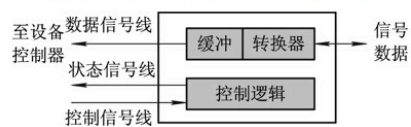
### • 设备控制器的功能：

- (1) 接收和识别命令（与CPU）
- (2) 数据交换（与CPU、设备）
- (3) 地址识别（设备地址）
- (4) 标识和报告设备的状态
- (5) 数据缓冲
- (6) 差错控制

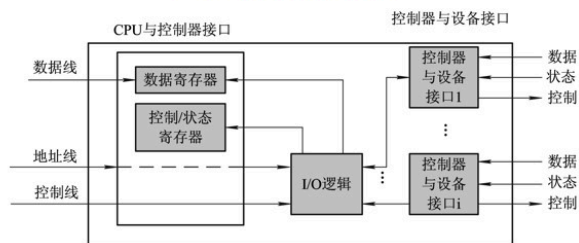
### • 设备控制器的组成：

- (1) 设备控制器与CPU的接口
- (2) 设备控制器与设备的接口
- (3) I/O逻辑（命令译码，地址译码）

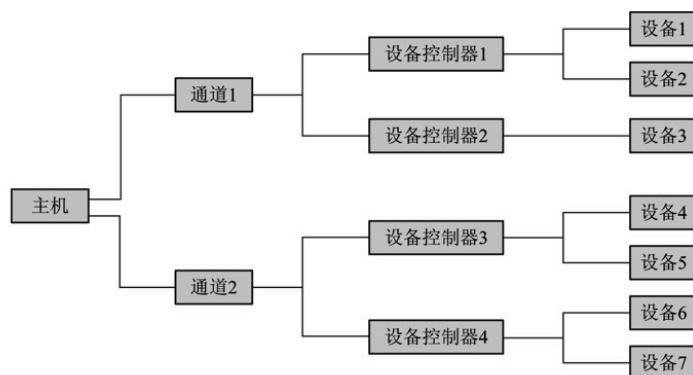
设备控制器与设备的接口模型



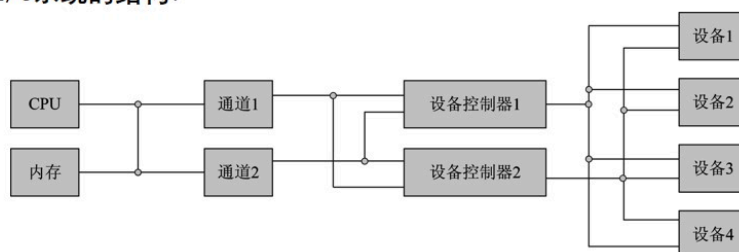
设备控制器组成示意



- 3. 通道 (I/O处理器)
- (1) 通道及通道与CPU之间的通信
- CPU与通道之间的关系是主从关系：CPU是主设备，通道是从设备。
- I/O系统的三级控制结构：
- 第一级CPU执行I/O指令启动通道，第二级通道接收I/O指令，执行通道程序发送给设备控制器，第三级设备控制器对设备完成I/O操作



- 通道价格高，配置较少，因此提出多通路通道。一台计算机中可以配置多个通道，一个通道可以连接多个设备控制器，一个设备控制器可以连接多台同类型的设备。可以缓解I/O瓶颈问题，个别通道阻塞时，可以使用其它通道。
- 多通路I/O系统的结构：



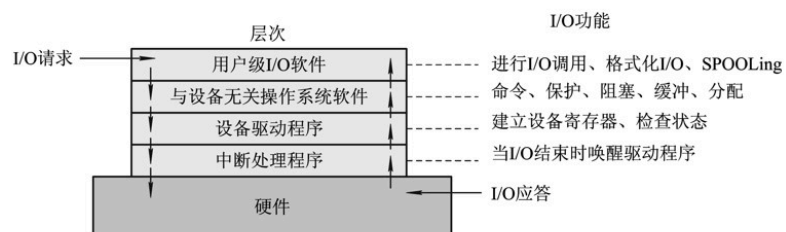
## （2）通道的类型

- ① 字节多路通道；以字节为单位按时间片轮转共享主通道。字节效率低，适用于低速设备，如打印机。
- ② 数组选择通道；以数据块为单位，一段时间只能服务一台设备。
- ③ 数组多路通道；先为一台设备执行通道命令，传送一批数据后自动转换设备。结合了以上的优点。

- I/O系统的软件组织

- 为了实现设备无关性的目标，操作系统把I/O软件组织由下至上分成以下层次：中断处理程序、设备驱动程序、与设备无关的I/O软件、用户级的I/O软件。

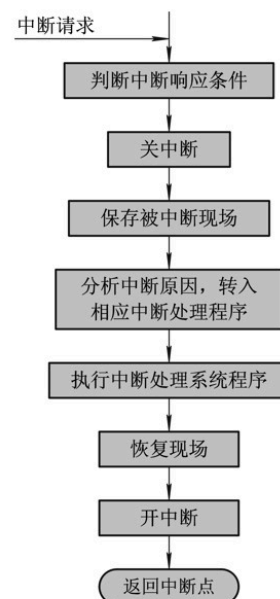
- I/O软件的层次结构图：



## I/O系统的软件组织

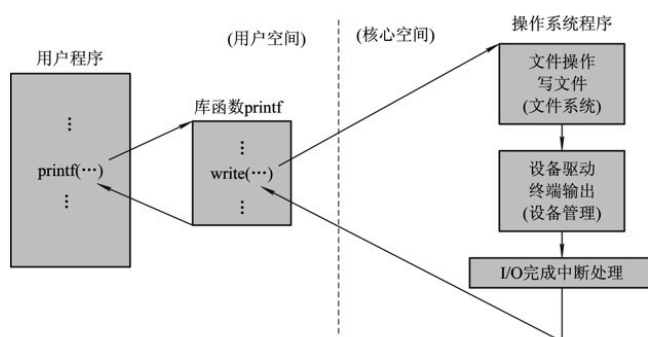
### • 1. 中断处理程序

- (1) 检查CPU响应中断的条件是否满足;
- (2) CPU响应中断后立即关中断; (不能响应其它中断)
- (3) 保存被中断进程的CPU现场; (寄存器信息)
- (4) 分析中断原因, 转入相应的设备中断处理程序;  
(识别中断类型, 时钟中断还是磁盘中断, 并判断优先级)
- (5) 执行中断处理程序;
- (6) 退出中断处理, 恢复被中断进程的CPU现场或调度新的进程执行;
- (7) 开中断, CPU继续执行。



## I/O系统的软件组织

- 2. 设备驱动程序
- 系统完成I/O请求的具体处理过程是：用户进程发出I/O请求→系统接受这个I/O请求→设备驱动程序具体完成I/O操作→I/O操作完成后用户进程继续执行。
- I/O请求处理过程示意：



## I/O系统的软件组织

- I/O请求的处理过程：
  - (1) 将抽象要求转换为具体要求；
  - (2) 检查I/O请求的合法性；
  - (3) 读出和检查设备状态；
  - (4) 传送必要的参数；
  - (5) 启动I/O设备；
  - (6) I/O完成。
- 设备驱动程序的功能：
  - (1) 将接收到的抽象命令（要求）转换为具体要求；
  - (2) 接受用户的I/O请求；
  - (3) 取出请求队列中队首请求，将相应设备分配给它；
  - (4) 处理来自设备的中断，及时响应由控制器或通道发来的中断请求，并根据其中断类型调用相应的中断程序进行处理。

- 3. 与设备无关的操作系统软件
- (1) 设备无关性概念
- 用户编写的应用程序独立于具体使用的物理设备（逻辑设备名），即使更换了物理设备应用程序也无须改变。
- 优点：
  - ① 设备分配更加灵活
  - ② 可以实现I/O重定向
- (2) 设备无关性软件
- 功能：
  - ① 向用户级I/O软件提供统一接口
  - ② 设备命名
  - ③ 设备保护
  - ④ 提供与设备无关的数据块大小
  - ⑤ 对独占设备的分配与回收
  - ⑥ 缓冲管理
  - ⑦ 差错控制

## I/O系统的软件组织

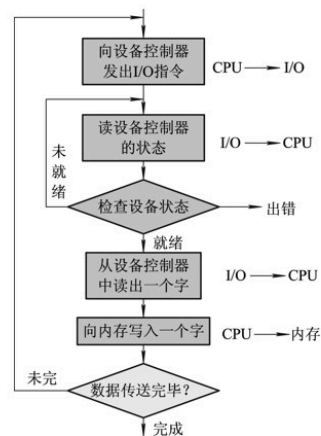
- 4. 用户级I/O软件
- 用户级I/O软件是I/O系统的最上层软件，负责与用户和设备无关I/O软件通信，即它面向用户。
- 用户级I/O软件主要包含I/O操作的库例程（库函数，printf）和SPooling系统。

## 5.4 I/O设备控制方式

### • 1.程序直接I/O控制方式

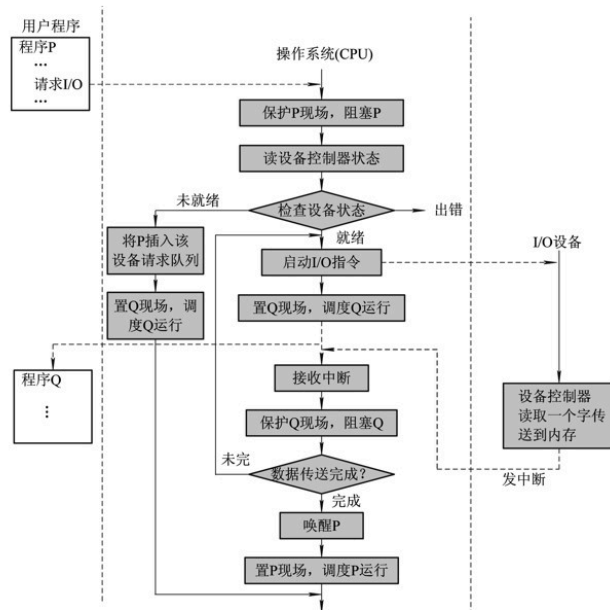
- 早期计算机，程序直接I/O控制方式由用户进程直接控制内存或CPU与外设之间的信息传送。CPU不断测试设备等待空闲，查询方式或忙-等方式。CPU资源浪费

- 直接I/O控制方式的控制流程：



## • 2. 程序中中断I/O控制方式

- 中断处理方式在一定程度上实现了CPU与外设并行，同时还可以实现多台外设之间的并行，从而提高了计算机系统的工作效率。
- 键盘输入字符100ms，字符写入内存0.1ms。直接I/O，CPU需要等待99.9ms。中断的话，只需要0.1ms。



### • 3. DMA控制方式

- 前面的方式数据传输以字节为单位，效率低。在块设备的I/O系统中，普遍采用直接存储器存取（Direct Memory Access, DMA）I/O控制方式，即DMA控制方式。
- 在该方式中，数据传送可以绕过CPU直接利用DMA控制器实现内存和外设之间的数据交换，每交换一次可以传送一个数据块。

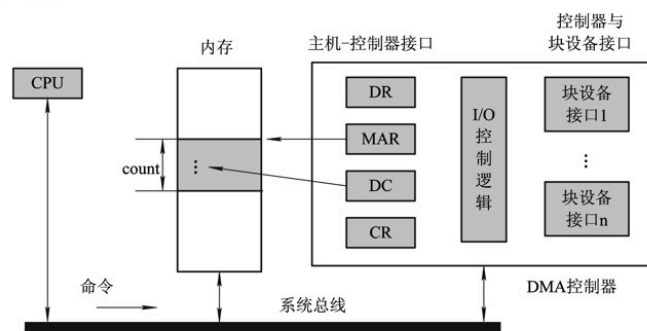
#### • 特点：

- （1）内存与设备之间以**数据块**为单位进行数据传输，即每次至少传输一个数据块。
- （2）DMA控制器获得总线控制权而直接与内存进行数据交换，CPU不介入数据传输事宜。
- （3）CPU仅在数据块传送的开始和结束时进行干预，而数据块的传输和I/O管理均由DMA控制器负责。

#### • DMA方式有：

- （1）周期窃取方式；
- （2）直接存取方式；
- （3）数据块传送方式

- DMA控制器的组成

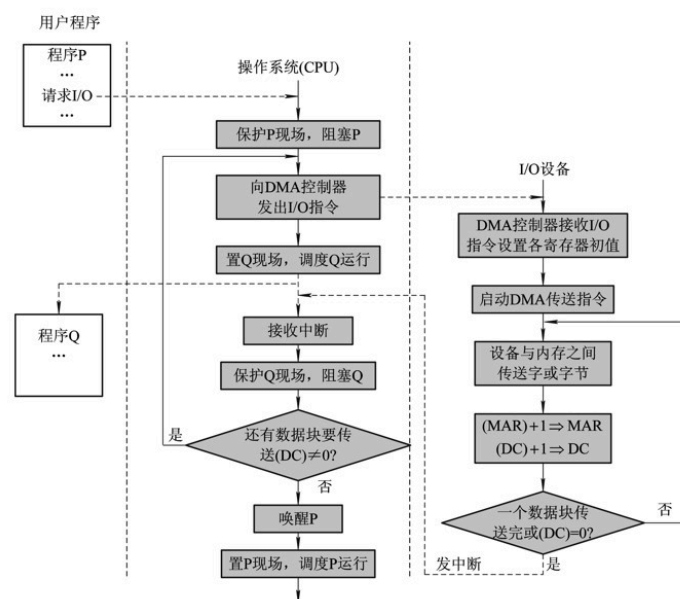


为了实现在主机与DMA控制器之间成块数据的直接交换，必须在DMA控制器中设置如下4类寄存器：

- (1) 命令/状态寄存器 (CR)：CPU命令，设备状态
- (2) 内存地址寄存器 (MAR)：设备地址，或内存起始地址
- (3) 数据寄存器 (DR)：暂存数据
- (4) 数据计数器 (DC)

- DMA控制方式的处理过程
- DMA控制方式传送数据的步骤如下：
  - (1) 进程请求I/O时，CPU就向DMA控制器发出一条I/O命令（运动会排方阵给班主任发指令），该命令被送至命令/状态寄存器；同时，CPU挪用一個系统总线周期（工作周期）将准备存放输入数据的内存起始地址（或准备输出数据的内存源地址），以及要传送的字节数分别存入内存地址寄存器和数据计数器，且将磁盘中的源地址（或目标地址）直接送入DMA控制器的I/O控制逻辑，然后启动DMA控制器进行数据传送。
  - (2) CPU将总线让给DMA控制器，由DMA控制器获得总线控制权来控制数据的传输。在DMA控制器控制数据传输期间，CPU不使用总线。（班主任带队排方阵）
  - (3) DMA控制器按照内存地址寄存器的指示，不断在设备与内存之间进行数据传输，并随时修改内存地址寄存器和数据计数器的值。当一个数据块传输完毕或数据计数器的值减少到0时（所有数据都已传输完毕），传输停止且向CPU发出中断信号。（方阵排好）
  - (4) CPU响应DMA控制器的中断请求，转向相应的中断处理程序进行后续处理。如果还有数据需要传输则按照相同方法重新启动剩余数据的传送。（给下一个班主任发指令）

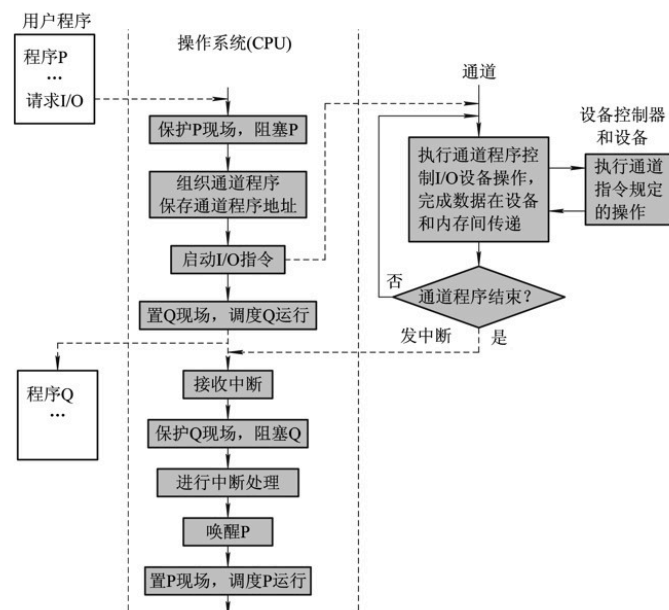
DMA控制方式处理过程示意图：



- 4.I/O通道控制方式

- 传输数据的步骤：
  - (1) 当进程提出I/O请求时，CPU对通道发出启动命令且传送相应参数，然后转向处理其他事务。（给班主任群里发个通知）
  - (2) 通道根据收到的启动命令调出通道程序执行，于是设备、通道、CPU并行工作。
  - (3) 通道逐条执行通道程序中的通道命令来指示设备完成规定的操作，控制完成设备与内存之间的数据传输。
  - (4) 数据传输完毕，通道向CPU发出中断请求。
  - (5) CPU响应通道提出的中断请求，对本次I/O进行结束处理。

- 通道控制方式处理过程：



## • 四种控制方式的对比

- 1 程序直接I/O控制方式（一个一个排方阵）：简单低效，适用于CPU速度慢且设备不多。（婴儿）
- 2 程序中断I/O控制方式（排一个，等的时候去排下一个）：适用于设备不多的时候。（幼儿）
- 3 DMA控制方式（每个班交给班主任）：减少了中断次数，但仍然依赖CPU，效率依然不高。（少儿）
- 4 I/O通道控制方式（交给班主任群）：效率高，但通道价格高，经济性低。（青年）

**单选题** 1分

如果I/O设备与存储设备进行数据交换而不经CPU来完成，这种数据交换方式是\_\_\_\_\_。

- ☐ A 程序查询
- ☐ B 中断方式
- ☐ C DMA方式
- ☐ D 无条件存取方式

**单选题** 1分

在I/O设备控制方式的发展过程中，最主要的推动力是\_\_\_\_\_。

- ☐ A 提高资源利用率
- ☐ B 提高系统吞吐量
- ☐ C 减少CPU对I/O控制的干预
- ☐ D 提高CPU和I/O设备并行操作的程度

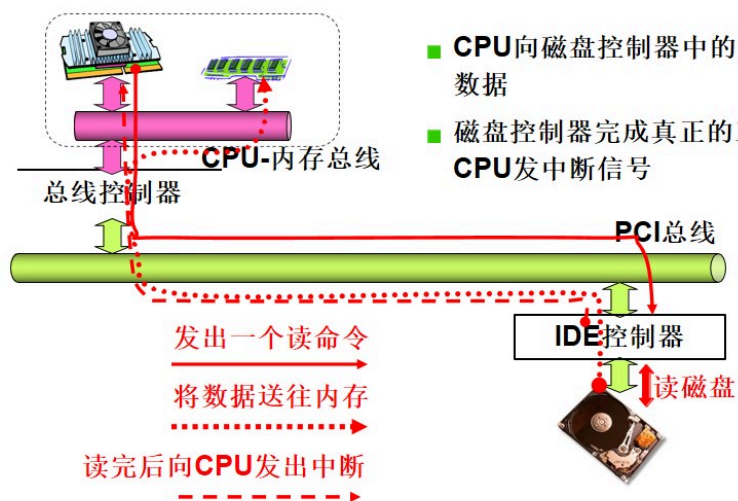
**单选题** 1分

在以下I/O控制方式中，需要CPU干预最少的是\_\_\_\_\_。

- ☐ A 程序I/O方式
- ☐ B 中断控制方式
- ☐ C DMA控制方式
- ☐ D 通道控制方式

## 5.5 生磁盘的使用

仍然从硬件开始...

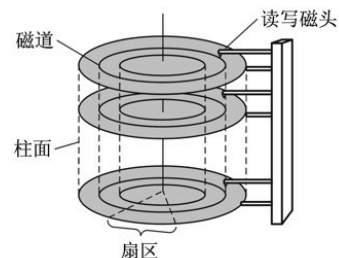


- CPU向磁盘控制器中的寄存器读写数据
- 磁盘控制器完成真正的工作，并向CPU发中断信号



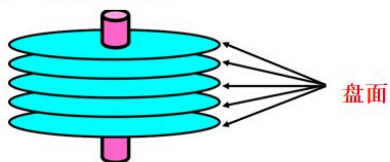
## • 存储设备概述

- 1. 硬盘
- 优点：容量大、存取速度快
- **硬盘**属于直接存取设备（也称随机存取设备），它的每个物理盘块具有唯一的地址，允许系统直接存取磁盘上的任意一个物理盘块。
- 2. 磁带
  - 磁带是一种典型的顺序存取设备。
  - 优点：存储容量大、文件卷可拆卸、便于保存等。
- 3. 光盘
  - 随机存取设备。
  - 优点：容量大、存取速度快，防潮、防磁、防震、便于长期保存、价格便宜等。
- 4. 闪存
  - 可移动存储介质
  - 优点：可反复读写、支持随机存取、寿命长、可靠性高、使用方便、没有任何机械运动部件、读写速度比较快，使用时不需要额外电源及存储密度高等。

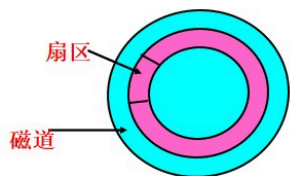


## 使用磁盘从认识磁盘开始

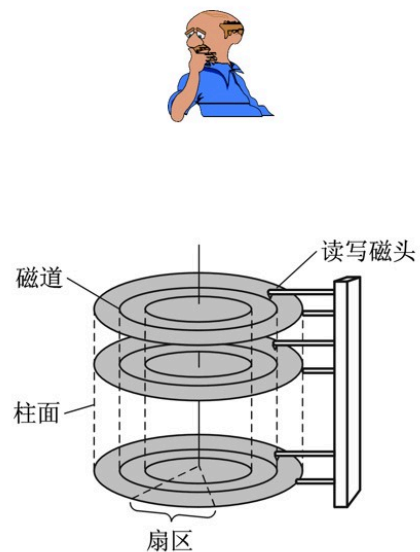
- 画一个示意图:



- 看看俯视图:



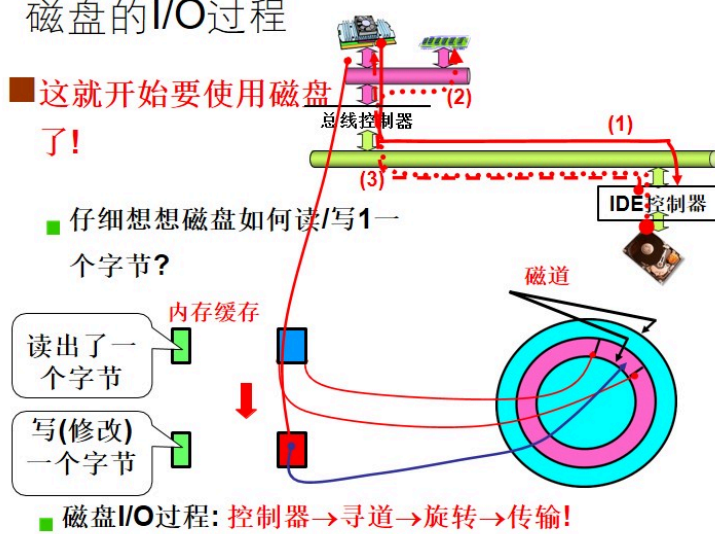
- 磁盘的访问单位是扇区
- 扇区大小: **512字节**
- 扇区的大小是传输时间和碎片浪费的折衷



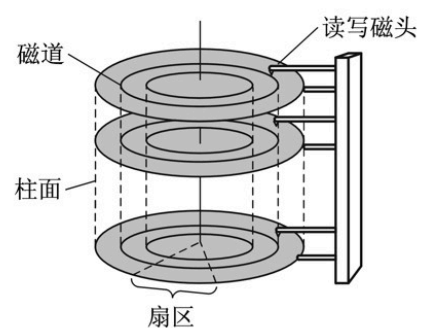
## 磁盘的I/O过程

■ 这就开始要使用磁盘了!

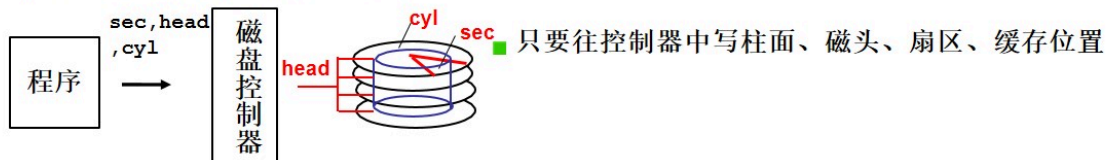
■ 仔细想想磁盘如何读/写1一个字节?



■ 磁盘I/O过程: 控制器→寻道→旋转→传输!

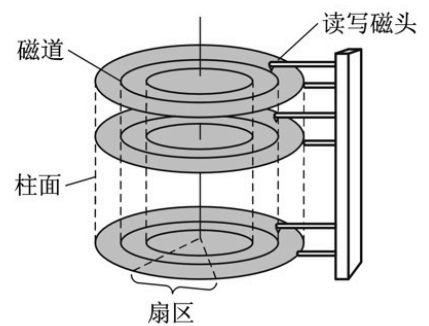


## 最直接的使用磁盘

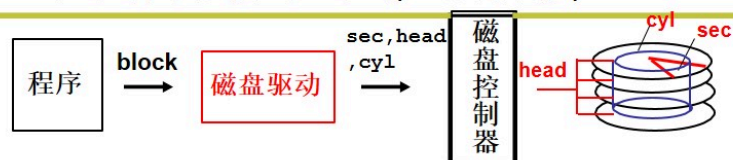


```
void do_hd_request(void)
{
    ...hd_out(dev, nsect, sec, head, cyl, WIN_WRITE, ...);
    port_write(HD_DATA, CURRENT->buffer, 256);
}
```

```
void hd_out(drive, nsect, sec, head, cyl, cmd...)
{
    port = HD_DATA; // 数据寄存器端口 (0x1f0)
    outb_p(nsect, ++port); outb_p(sec, ++port);
    outb_p(cyl, ++port); outb_p(cyl >> 8, ++port);
    outb_p(0xA0 | (drive << 4) | head, ++port);
    outb_p(cmd, ++port);
}
```



## 通过盘块号读写磁盘(一层抽象)



- 磁盘驱动负责从block计算出cyl, head, sec(CHS)

问题：如何编址？为什么这样编址？ **block相邻的盘块可以快速读出**

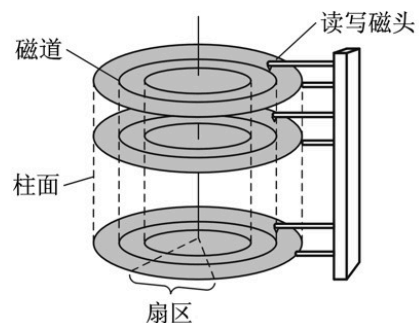
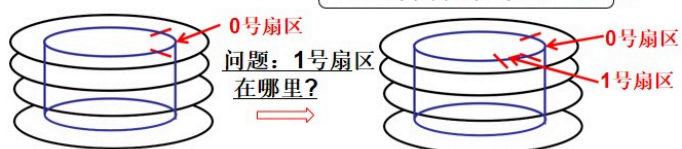
磁盘访问时间 = 写入控制器时间

50M/秒,约0.3ms

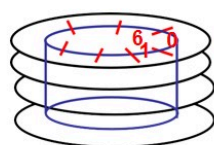
寻道时间 + 旋转时间 + 传输时间

12 ms to 8 ms

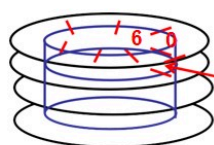
7200转/分钟:半周 4 ms



## 从CHS到扇区号，从扇区到盘块



问题：接下来呢



7号扇区

问题：C、H、S得到的扇区号是？  $C \times (\text{Heads} \times \text{Sectors}) + H \times \text{Sectors} + S$

磁盘访问时间 = 写入控制器时间

共计约10ms

寻道时间 + 旋转时间 + 传输时间

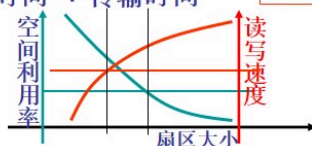
扇区大小固定，但操作系统可以每次读/写连续的几个扇区(盘块)

■ 从扇区到盘块：每次读写1 K：

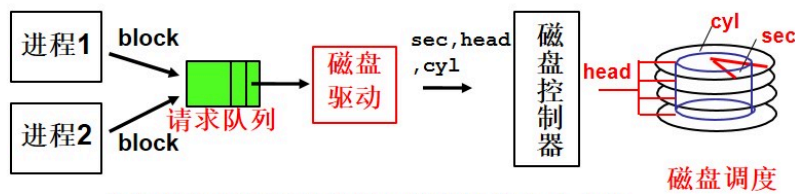
碎片0.5K；读写速度100K/秒；

每次读写1 M：碎片0.5M；读

写速度约40M/秒



## 多个进程通过队列使用磁盘(第二层抽象)



- 多个磁盘访问请求出现在请求队列怎么办？
- 调度的目标是什么？调度时主要考察什么？

目标当然是平均  
访问延迟小！

寻道时间是主要  
矛盾！

- 给调度算法，仍然从**FCFS**开始...

## 1.FCFS磁盘调度算法

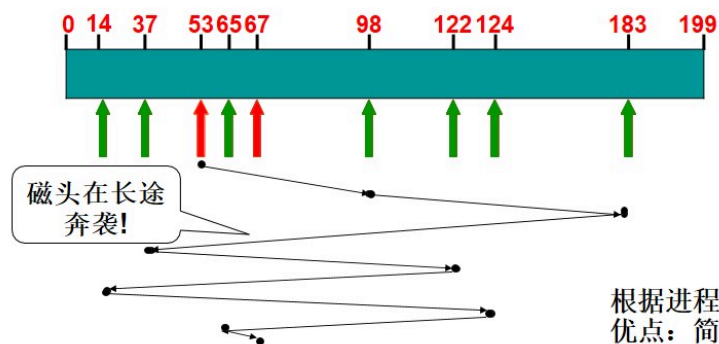
### ■ 最直观、最公平的调度：

■ 一个实例：磁头开始位置=53；

请求队列=98, 183, 37, 122, 14, 124, 65, 67

FCFS: 磁头共  
移动640磁道!

在移动过程中把经过  
的请求处理了!



根据进程请求访问磁盘的先后次序进行调度。  
优点：简单、公平，每个进程的访问请求都能依次得到处理，不会出现某个进程的访问请求长时间得不到满足的情况。

**填空题** 2分

假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试求采用FCFS算法时，系统服务的顺序、磁头移动的距离 [填空1] 以及磁头改变方向的次数 [填空2] 。

- 假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试求采用FCFS算法时，系统服务的顺序、磁头移动的距离以及磁头改变方向的次数。
- 【解答】 服务顺序：90→55→72→100→88→93→66
- 移动距离：35+17+28+12+5+27=124（跨越的柱面总数）
- 磁头改变方向的次数：4次。

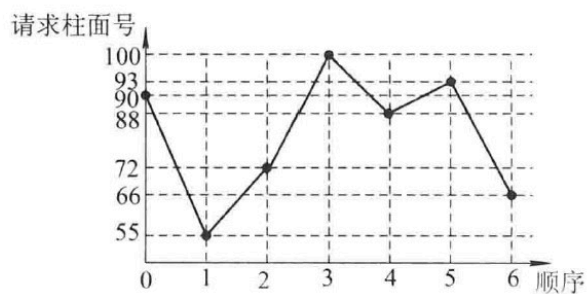


图 5-26 FCFS 移臂调度算法示例图

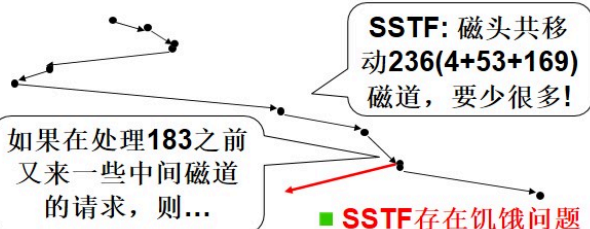
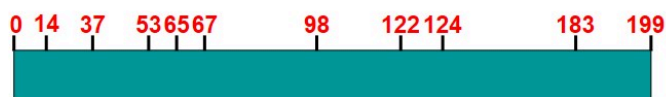
## 2. 最短寻道时间优先算法SSTF

根据请求进程要访问的磁道离当前磁头位置的远近来决定调度的顺序，近者先调度。保证了每次寻道距离最短，没有保证平均寻道距离最短。

### ■ Shortest-seek-time First:

■ 继续该实例：磁头开始位置=53；

请求队列=98, 183, 37, 122, 14, 124, 65, 67



■ SSTF存在饥饿问题（出租车）

**填空题** 2分

假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试用SSTF算法求系统服务的顺序、磁头移动的距离 **[填空1]** 以及磁头改变方向的次数 **[填空2]** 。

假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试用SSTF算法求系统服务的顺序、磁头移动的距离以及磁头改变方向的次数。

【解答】 服务顺序：90→88→93→100→72→66→55

移动距离：2+5+7+28+6+11=59（跨越的柱面总数）

磁头改变方向的次数：2次。

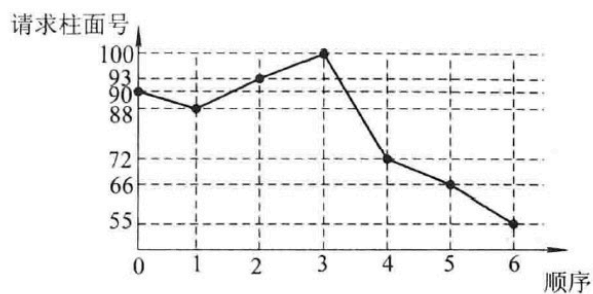


图 5-27 SSTF 移臂调度算法示例图

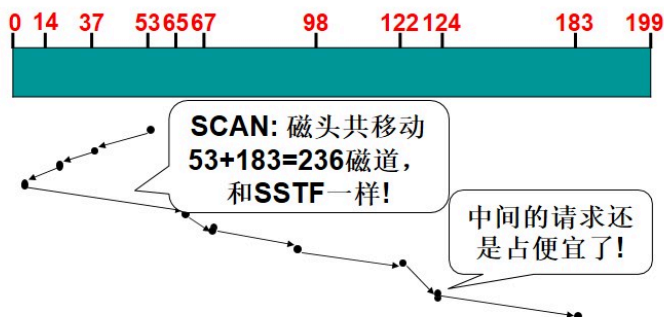
### 3. 扫描算法SCAN

减少磁头改变方向的一类调度算法，思想是：首先考虑磁盘请求的磁头移动方向，在方向一致的情况下优先调度与当前磁头最近的请求。

■ **SSTF+中途不回折**：一直扫描到最外层或最内层，每个请求都有处理机会

■ 继续该实例：磁头开始位置=53；

请求队列=98, 183, 37, 122, 14, 124, 65, 67



**填空题** 2分

假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试用SCAN算法求系统服务的顺序、磁头移动的距离 **[填空1]** 以及磁头改变方向的次数 **[填空2]** 。

假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试用SCAN算法求系统服务的顺序、磁头移动的距离以及磁头改变方向的次数。

【解答】 服务顺序：90→88→72→66→55→0→93→100

移动距离：2+16+6+11+55+93+7=190（跨越的柱面总数）

磁头改变方向的次数：1次。

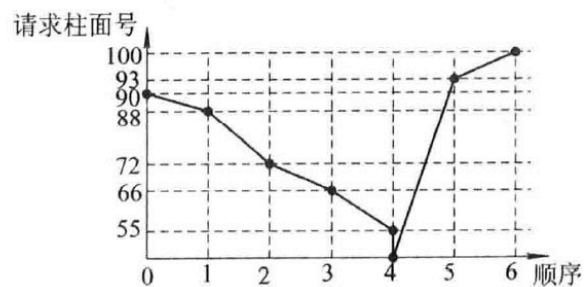


图 5-28 SCAN 移臂调度算法示例图

#### 4. 电梯算法

电梯算法 (Elevator Algorithm) 是对扫描算法的一种改进，其基本思想是：没有访问请求时磁头不动，有访问请求时磁头来回扫描，每次选择磁头移动方向上离当前磁头位置最近的访问请求进行处理；扫描过程中若磁头移动方向上仍有访问请求，则继续向同一个方向扫描；当磁头移动方向上不存在访问请求时则改变方向向相反方向扫描。

电梯算法和扫描算法的区别：来回扫描，不到最底层。

**填空题** 2分

假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试用电梯算法求系统服务的顺序、磁头移动的距离 [填空1] 以及磁头改变方向的次数 [填空2] 。

#### 4. 电梯算法

和扫描算法的区别：来回扫描，不到最底层

假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试用电梯算法求系统服务的顺序、磁头移动的距离以及磁头改变方向的次数。

【解答】 服务顺序：90→88→72→66→55→93→100

移动距离：2+16+6+11+38+7=80（跨越的柱面总数）

磁头改变方向的次数：1次。

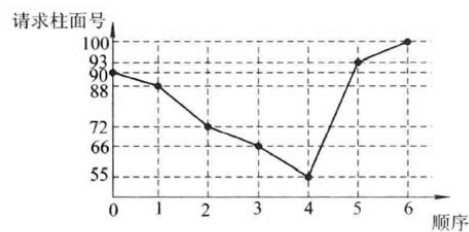


图 5-29 电梯移臂调度算法示例图

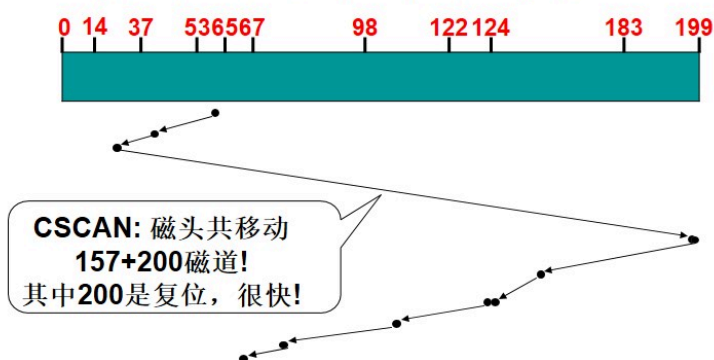
## 5. 循环扫描算法C-SCAN

与电梯算法的不同：是将来回扫描改为单向扫描，即没有访问请求时磁头不动，有访问请求时向一个方向扫描。当移动到最小或最大的磁道后，立即返回到最大或最小的磁道，中间不响应任何请求。

### ■ SCAN+直接移到另一端：两端请求都能很快处理

■ 继续该实例：磁头开始位置=53；

请求队列=98, 183, 37, 122, 14, 124, 65, 67



**填空题** 2分

假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试用CSCAN算法求系统服务的顺序、磁头移动的距离 [填空1] 以及磁头改变方向的次数 [填空2] 。

例：假设某磁盘的一组I/O操作的柱面号访问请求顺序依次是：55、72、100、88、93和66，当前磁盘位于90号柱面。试用CSCAN算法求系统服务的顺序、磁头移动的距离以及磁头改变方向的次数。

【解答】 服务顺序：90→88→72→66→55→100→93

移动距离：2+16+6+11+45+7=87（跨越的柱面总数）

磁头改变方向的次数：2次。

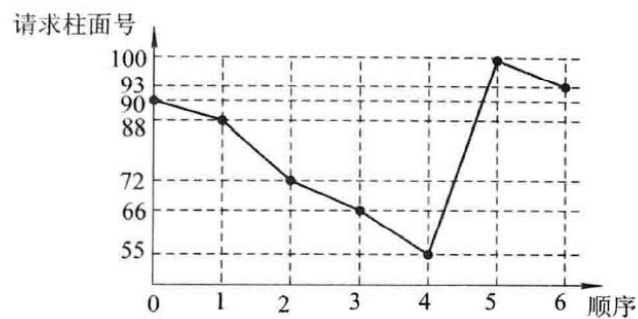
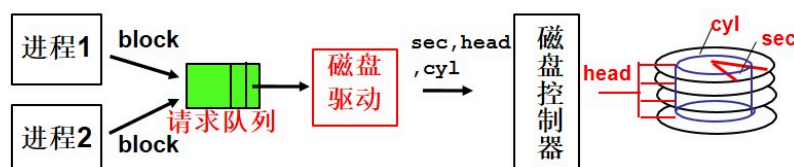


图 5-30 CSCAN 移臂调度算法示例图

## 生磁盘(raw disk)的使用整理



- (1) 进程“得到盘块号”，算出扇区号(sector)
- (2) 用扇区号make req，用电梯算法add\_request
- (3) 进程sleep\_off
- (4) 磁盘中断处理 唤醒进程!
- (5) do\_hd\_request算出cyl,head,sector
- (6) hd\_out调用out(...)完成端口写

**单选题** 1分

**磁盘调度算法的目标是什么？**

- ☐ A 提高磁盘存储容量
- ☐ B 提高磁盘的传输速率
- ☐ C 提高磁盘的访问速度
- ☐ D 提高磁盘的可靠性

**单选题** 1分

**【考研真题】**有一个长度为3200B的流式文件要存储在磁盘上，磁盘的每块可以存放512B，则存放该文件至少需要（ ）块磁盘。

- ☐ A 5
- ☐ B 6
- ☐ C 7
- ☐ D 3000

**单选题** 1分

对于以扇区（512B）为基本分配单位的FAT12文件系统（FAT的表项为12位），其所支持的磁盘分区容量为（ ）B。

- ☐ A 256K
- ☐ B 4M
- ☐ C 2M
- ☐ D 8M

## 5.6 从生磁盘到文件

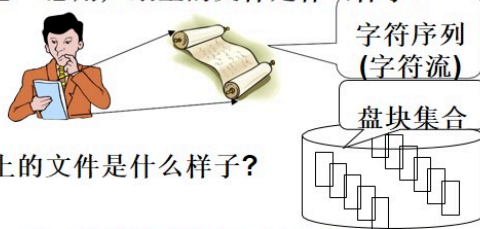
为什么引入文件？  
—**cooked disk**

## 引入文件，对磁盘使用的第三层抽象

■ **让普通用户使用raw disk:** 许多人连扇区都不知道是什么?要求他们根据盘块号来访问磁盘...

■ 需要在盘块上引入更高层次的抽象概念! 文件

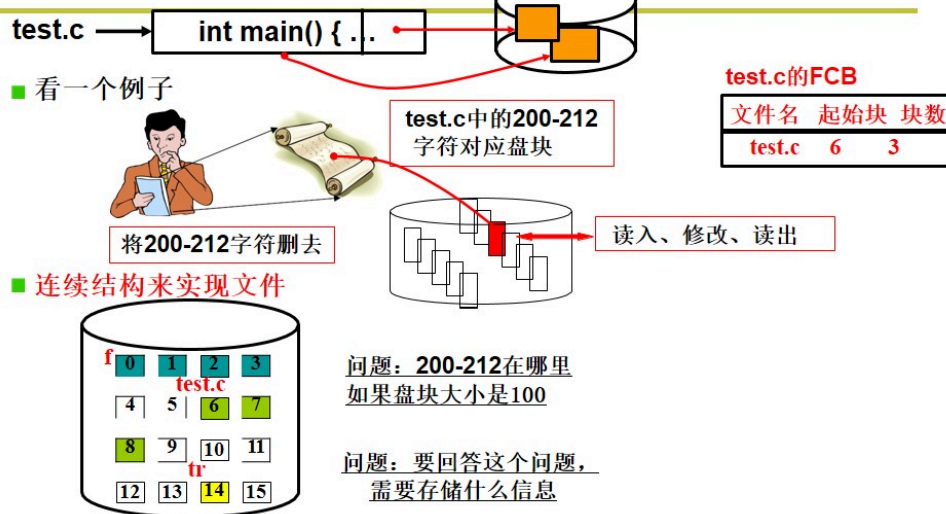
■ 首先想一想用户眼里的文件是什么样子?



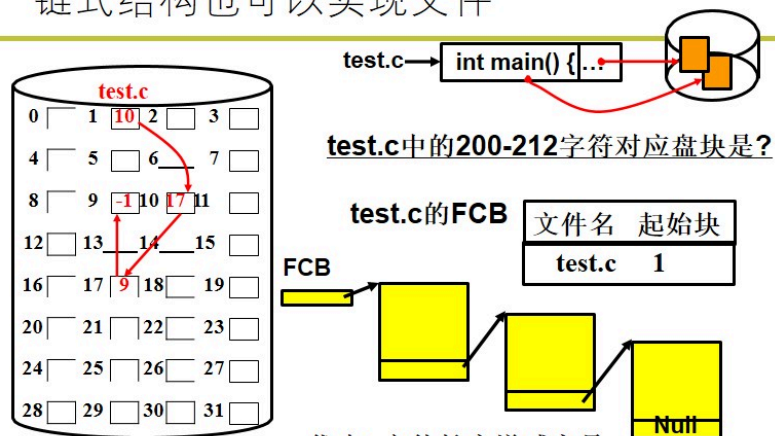
■ 磁盘上的文件是什么样子?

■ **文件:** 建立字符流到盘块集合的映射关系

## 映射的作用



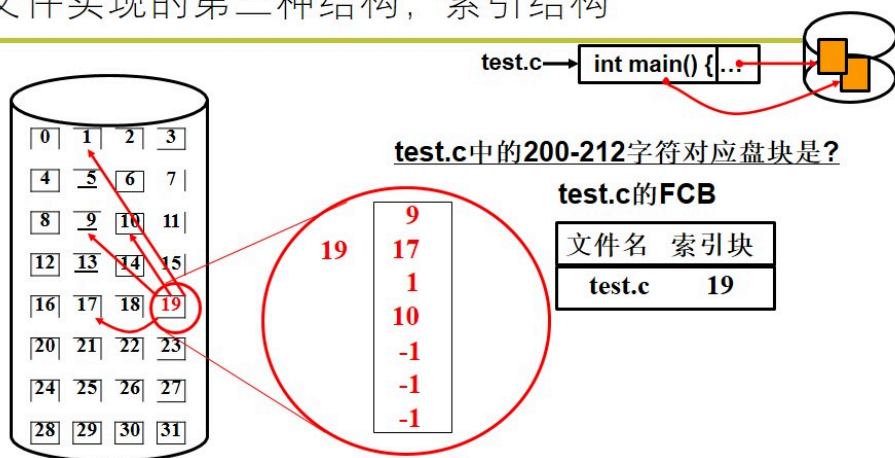
## 链式结构也可以实现文件



■ 优点: 文件长度增减容易

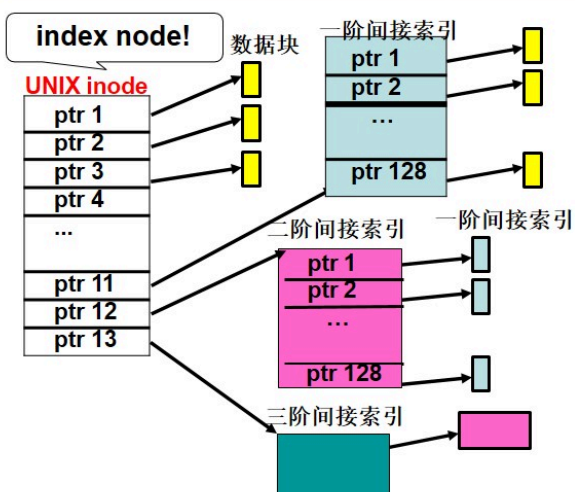
■ 缺点: 访问慢、可靠性差

### 文件实现的第三种结构，索引结构



■ 是连续和链式分配的有效折衷，  
在实际操作系统中较常用！

## 实际系统是多级索引



- 可以表示很大的文件
- 很小的文件高效访问
- 中等大小的文件访问速度也不慢!

这就是通用操作系统的魅力!

**单选题** 1分

I/O中断是CPU与通道协调工作的一种手段，所以在\_\_\_\_时就要产生中断。

- ☐ A CPU执行启动I/O指令而被通道拒绝接受
- ☐ B 通道接收了CPU的启动请求
- ☐ C 通道完成了通道程序的执行
- ☐ D 通道在执行通道程序的过程中

**单选题** 1分

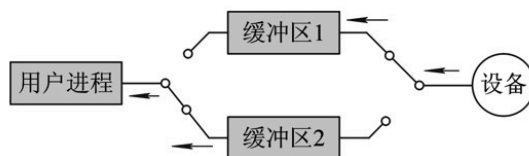
打印机适合于连接到\_\_\_\_\_。

- ☐ A 数组多路通道
- ☐ B 字节多路通道
- ☐ C 数组选择通道
- ☐ D A ~ C都不对

## 5.4缓冲技术与虚拟设备技术

### • 缓冲技术

- 解决CPU与IO速度不匹配问题
- 减少中断次数
- 解决数据瓶颈问题
- 实现缓冲通常有两种方法：一种是采用专门的硬件缓冲器，如设备控制器中的数据缓冲寄存器；另一种是在内存中开辟一段存储区作为缓冲区。
- 1. 单缓冲和双缓冲
- **单缓冲**是指在CPU与设备之间设置一个缓冲区。无法并行操作。（P-C）
- **双缓冲**又称缓冲对换，是在CPU与设备之间设置两个缓冲区。可以并行操作。

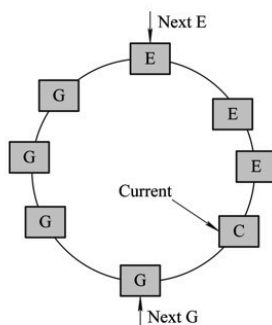


## 缓冲技术

### 2. 循环缓冲（当速度差距太远，双缓冲也无法满足）

循环缓冲中的每个缓冲区大小相同，并且根据缓冲区的当前状态将它们划分成3种类型：

- (1) 空缓冲区（记为E），用于装入数据。Next-E 下次生产者进程指针
- (2) 满缓冲区（记为G），已装满数据。Next-G 下次消费者进程指针
- (3) 当前工作缓冲区（记为C），当前正在使用的缓冲区。消费者当前指针



## 缓冲技术

以上为专用缓冲。当设备增多时，专用缓冲利用率不高。

### 3. 缓冲池

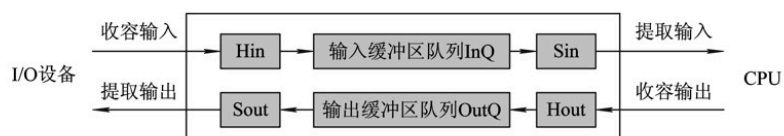
公用缓冲池技术是指缓冲池中的缓冲区可以供各种生产者和消费者共享。

空缓冲区队列；

输入缓冲区队列；

输出缓冲区队列。

缓冲池中缓冲区的工作方式:收容输入 (Hin)、提取输入 (Sin)、收容输出 (Hout)、提取输出 (Sout)。



## 缓冲技术

### 4. 高速缓存

高速缓冲存储器即高速缓存（Cache），是可以保存数据副本的高速存储器，访问高速缓存要比访问内存中的原始数据效率更高，速度更快。

高速缓存和内存缓冲区都介于高速设备和低速设备之间，区别是：

- （1）两者存放的数据不同：高速缓存放的低速设备的数据副本。缓冲区放的低速设备需要交换的数据。
- （2）两者的目的不同：高速缓存放副本避免频繁访问低速设备。缓冲区为了让高速设备与低速设备进行通信。

## • 虚拟设备技术

- 虚拟设备——用一个物理设备模拟出的多个逻辑上存在的设备。
- 虚拟设备通常采用SPooling技术实现。
- SPooling技术又称假脱机操作技术，是在联机情况下实现脱机I/O功能的，即用一道程序模拟外围输入控制机，将用户进程需要的数据从慢速设备预先输入到磁盘上，当用户进程需要数据时再从磁盘上直接读出；
- 用另一道程序模拟外围输出控制机，用户进程将输出的数据先传输到磁盘上，然后再由程序模拟的外围输出机将暂存在磁盘上的数据在慢速设备上逐个字符地输出。
- 硬件上，系统必须配置大容量磁盘使CPU与设备并行工作；软件上，操作系统必须采用多道程序设计技术。

## 虚拟设备技术

SPooling系统组成:

硬件: 在磁盘上划出两块存储区域: 输入井、输出井;

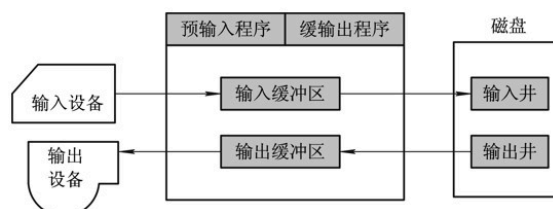
内存中开辟两个缓冲区: 输入缓冲区、输出缓冲区。

软件: (1) 预输入程序; (2) 缓输出程序; (3) 井管理程序

虚拟打印机: 需要打印时, 输出井创建空闲区, 放入打印数据, 建立请求打印表, 排入Spooling打印队列, 从井里取出数据, 放入缓冲区, 进行打印。不直接操作IO设备, 放入存储区按顺序输出, 不会争夺和死锁。

SPooling系统具有如下特点:

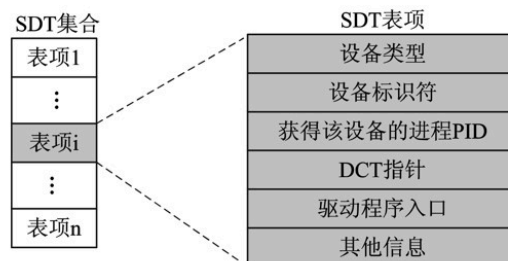
- (1) 提高了I/O的速度
- (2) 将独占设备改造成共享设备
- (3) 实现了虚拟设备功能



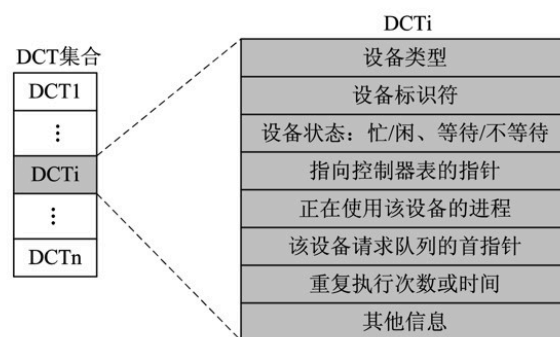
## 5.5 设备的分配与回收

### • 用于设备分配的数据结构

- 主要有系统设备表（SDT）、设备控制表（DCT）、控制器控制表（COCT）和通道控制表（CHCT）。
- 1. 系统设备表
- 记录物理设备相关信息



- 2. 设备控制表
- 记录设备类型，状态，控制器连接。



- 3. 控制器控制表COCT和通道控制表CHCT
- 记录设备控制器和通道的状态，连接情况及其他控制信息。

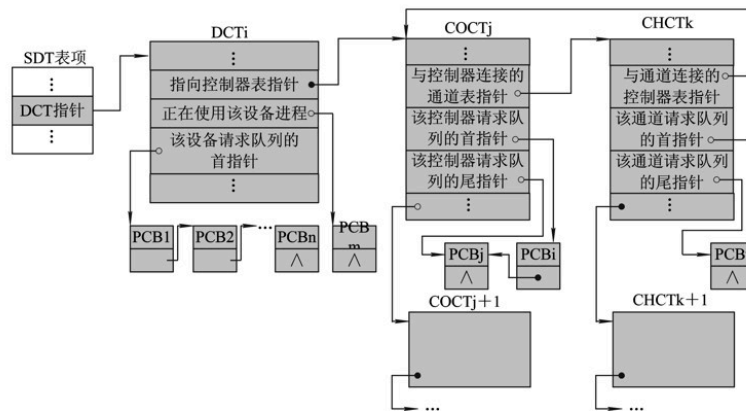
|              |
|--------------|
| 控制器标识符       |
| 控制器状态：忙/闲    |
| 与控制器连接的通道表指针 |
| 该控制器请求队列的首指针 |
| 该控制器请求队列的尾指针 |
| 其他信息         |

(a) 控制器控制表(COCT)

|              |
|--------------|
| 通道标识符        |
| 通道状态：忙/闲     |
| 与通道连接的控制器表指针 |
| 该通道请求队列的首指针  |
| 该通道请求队列的尾指针  |
| 其他信息         |

(b) 通道控制表(CHCT)

• 4. SDT、DCT、COCT和CHCT之间的连接关系



## 5.5 设备的分配与回收

### • 设备分配

- 1. 设备分配的原则和策略
- 总原则是：
  - (1) 充分发挥设备的使用效率，尽可能地让设备忙起来。
  - (2) 避免由于不合理的分配而造成死锁的发生。
  - (3) 将用户程序和具体物理设备相隔离。用户程序面对的是逻辑设备，设备分配程序完成把逻辑设备转换为对应的物理设备，再根据物理设备号进行分配工作。
- 分配策略：
  - (1) 对独占设备采用独占分配策略
  - (2) 对共享设备采用共享分配策略
  - (3) 对虚拟设备采用虚拟分配策略

## 设备分配

### 2. 设备分配方式

- (1) 设备的静态分配：指在用户进程开始执行之前，操作系统就将该进程所需要的全部设备、设备控制器和通道一次性地分配给它。
- (2) 设备的动态分配：指进程在运行过程中需要I/O时再由操作系统为其分配设备。

### 3. 设备分配算法

- (1) 先来先服务算法：当多个进程对同一个设备提出I/O请求时，系统按照提出I/O请求的先后顺序将所有请求进程排成一个队列（称为设备请求队列或设备队列），设备分配程序总是将空闲设备分配给位于队首的进程。
- (2) 优先级高者优先算法：根据任务的紧迫程度赋予进程一个优先级，当多个进程对同一个设备提出I/O请求时，系统根据进程的优先级大小将所有请求进程排成设备请求队列，设备分配程序总是将空闲设备分配给位于队首的当前优先级最高的进程。

## 设备分配

### 4. 设备独立性

设备独立性又称设备无关性，指用户程序独立于具体使用的物理设备。

好处：用户程序与具体物理设备无关，系统增减或变更物理设备时程序不必修改，也易于对系统中的设备故障进行处理。

逻辑设备表 (1)

| 逻辑设备名        | 物理设备名 | 驱动程序入口地址 |  | 逻辑设备名        | SDT表项指针 |
|--------------|-------|----------|--|--------------|---------|
| /dev/tty1    | 3     | 20420    |  | /dev/tty1    | 4       |
| /dev/tty2    | 4     | 20A00    |  | /dev/tty2    | 5       |
| /dev/printer | 7     | 21300    |  | /dev/printer | 8       |
| ...          | ...   | ...      |  | ...          | ...     |

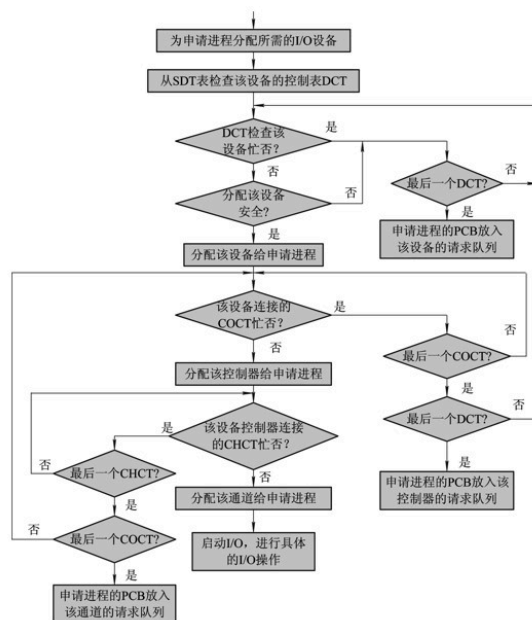
逻辑设备表 (2)

## 设备分配

### 5. 设备分配的实施

#### 步骤:

- (1) 分配设备；根据算法分配
- (2) 分配设备控制器；从设备控制表DCT找到指针
- (3) 分配通道；通过控制器控制表COCT找到与设备控制器相连的通道指针



## 5.5 设备的分配与回收

### • 设备回收

回收算法：

- （1）根据设备类型查系统设备表（SDT）和设备控制表（DCT）。
- （2）查DCT、COCT（控制器控制表）和CHCT（通道控制表），找到这样的DCT、COCT和CHCT：其“当前进程”为释放进程。如果相应的DCT、COCT和CHCT的请求队列不空则转（3），否则转（4）。
- （3）取出相应请求队列的第一个进程，把设备或设备控制器或通道分配给该进程，分配算法如前所述。
- （4）因为没有等待该设备、设备控制器和通道的进程，所以置设备、设备控制器和通道状态为空闲，同时修改SDT表中该设备的表项内容。

**单选题** 1分

为了使多个进程能有效地同时处理输入和输出，最好使用\_\_\_\_结构的缓冲技术。

- ☐ A 缓冲池
- ☐ B 循环缓冲
- ☐ C 单缓冲
- ☐ D 双缓冲

**单选题** 1分

在采用SPOOLing技术的系统中，用户的打印数据首先被送到\_\_\_\_\_。

- ☐ A 内存固定区域
- ☐ B 磁盘固定区域
- ☐ C 终端
- ☐ D 打印机

**填空题** 1分

**(考研真题)** 某磁盘的转速为10 000r/min，平均寻道时间为6ms，磁盘传输速率为20MB/s，磁盘控制器时延为0.2ms，读取一个4KB 的扇区所需的平均时间约为多少？ **[填空1]**

磁盘转速是10 000r/min，平均转1转的时间是6ms，因此查询扇区的平均时间是 $(1/2) \times 6\text{ms} = 3\text{ms}$ 。因为磁盘传输速率是20MB/s，所以读取4KB扇区信息的时间为 $4\text{KB} / (20\text{MB/s}) = 0.2\text{ms}$ 。时延为0.2ms，平均寻道时间为6ms，因此总时间为 $3\text{ms} + 6\text{ms} + 0.2\text{ms} + 0.2\text{ms} = 9.4\text{ms}$ 。

**主观题** 10分

**(考研真题)** 假设有11个进程先后提出磁盘I/O请求，当前磁头正在110号磁道处，并预向磁道序号增加的方向移动。请求队列的顺序为30、145、120、78、82、140、20、42、165、65、55，分别用FCFS调度算法和SCAN调度算法完成上述请求，写出磁道访问顺序和每次磁头移动的距离，并计算平均移动磁道数。

(1)FCFS调度算法：访问顺序为30、145、120、78、82、140、20、42、165、55、65；  
移动距离为80、115、25、42、4、58、120、22、123、110、10；  
平均移动磁道数为  $(80+115+25+42+4+58+120+22+123+110+10) / 11=64.45$ 。

(2)SCAN调度算法：访问顺序为120、140、145、165、82、78、65、55、42、30、20；  
移动距离为10、20、5、20、83、4、13、10、13、12、10；  
平均移动磁道数为  $(10+20+5+20+83+4+13+10+13+10+10) / 11=18.18$ 。

# 第五章（完）